

Lab 4 Extra

准备工作：创建并切换到 lab4-extra 分支

请在自动初始化分支后，在开发机依次执行以下命令：

```
$ cd ~/学号
$ git fetch
$ git checkout lab4-extra
```

初始化的 lab4-extra 分支基于课下完成的 lab4 分支，并且在 tests 目录下添加了 lab4_dirty_page_sample 样例测试目录。

提示：本题测试点独立性相对较高，且不依赖样例。若未完全实现，也可以提交以获得部分分数。

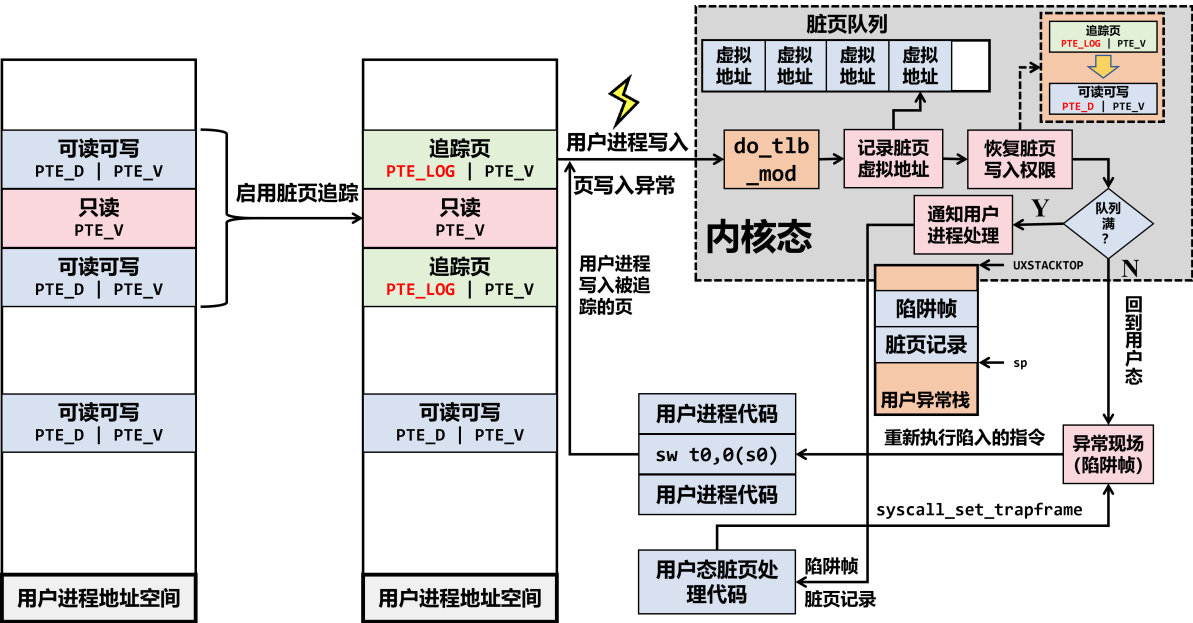
题目背景

在现代操作系统中，内存管理不再局限于物理页的分配与映射，正转向对程序内存访问行为的精细化监控与追踪。其中，“脏页追踪”是一项被广泛应用的底层技术。

所谓“脏页”，是指在某一段特定时间内，被应用程序写入或修改过的内存页面。掌握脏页记录意味着系统能精确追踪内存的变动，这种机制在一些领域有所应用：在云计算场景中，虚拟机热迁移时，它用于获取迁移期间的内存修改，并同步至目标机器；而在后续的文件系统实验中，它可以用于识别缓存中的修改内容，确保在关闭文件时能准确地将数据持久化到磁盘。

在本题目中，你需要结合 Lab4 课下实验中的系统调用机制、写时复制机制、页写入异常等知识点，在 MOS 中实现脏页记录功能。

要求实现的功能如下图所示，你可以右键单击该图片，选择“在新标签页中打开”，以便于随时查看。



具体地：

1. 用户进程首先使用“启用脏页追踪”的系统调用，指定自身地址空间内的一段地址范围启用脏页追踪，将其中的可写页面打上 PTE_LOG 的追踪标志位，同时去除其 PTE_D 可写标志位。这样，当用户进程尝试写入被追踪的页面时，将引起页写入异常，使得内核能记录脏页；对于只读或者未映射的页面，则不进行操作；

2. 当用户进程写入追踪页时，将发生页写入异常，陷入内核态。内核在对应用户进程的脏页队列中记录对应页面的虚拟地址，并去掉 `PTE_LOG` 追踪标志位，恢复该页的可写标志位。这样，当从内核态返回用户进程，重新执行写入指令时，相应的页已经是可写状态，写入指令可成功执行；
3. 显然，内核为每个进程维护的脏页队列大小是有限的，当脏页队列满时，类似 Lab4 课下实现的写时复制机制，内核将控制权交由用户态的处理代码，并同时完整的脏页记录传递给该处理代码，处理代码可对脏页记录进行转储等操作，处理完毕后使用系统调用从处理逻辑回到异常现场。**注意：为避免用户态处理代码访问其他追踪页时引发嵌套异常，内核需将完整的脏页记录复制到用户态的异常栈上，供用户进程访问。复制完成后，内核将清空该进程的脏页队列。**

题目简介

注意：本节中的代码仅用于辅助说明，同学们无需复制，后续“题目要求”部分将给出更加详细的答题指示。

脏页记录数据结构

由于内核需要为每个用户进程记录脏页地址，在 `Env` 进程控制块添加字段，记录是否启用脏页记录、脏页队列、用户态脏页处理代码的地址。

```
#include <dirtyqueue.h>

struct Env {
    ...
    u_int log_enabled;
    struct DirtyPageQueue log_queue;
    u_long log_entry;
};
```

其中，`struct DirtyPageQueue` 是脏页队列数据结构，同学们无需自行实现，其 API 如下，在实现本题目要求的过程中，你只应通过下方的 API 操作脏页队列，不建议自行操作 `struct DirtyPageQueue` 结构体，使用如下函数时，需要包含 `dirtyqueue.h` 头文件（`#include <dirtyqueue.h>`）：

- `void dirty_init(struct DirtyPageQueue *queue)`：初始化脏页队列，用于在初始化进程控制块时，初始化其 `log_queue` 字段；
- `int dirty_is_full(struct DirtyPageQueue *queue)`：判断对应的脏页队列是否已满，若已满，返回 `1`，否则返回 `0`；
- `int dirty_is_empty(struct DirtyPageQueue *queue)`：判断对应的脏页队列是否为空，若为空，返回 `1`，否则返回 `0`；
- `void dirty_add(struct DirtyPageQueue *queue, u_long new_dirty_page)`：将脏页记录 `new_dirty_page`（新增脏页的虚拟地址，注意由于一个页面对应的是一个地址范围，此处应当传入对应页面的起始地址）加入队列。要求调用该函数时队列未滿；若队列已满，将 `panic`；
- `u_long dirty_remove(struct DirtyPageQueue *queue)`：从脏页队列中获取一个脏页记录（先进先出），并将该记录出队。要求调用该函数时队列不为空；若队列为空，将 `panic`；
- `void dirty_clear(struct DirtyPageQueue *queue)`：将脏页队列清空，即，删除其中的所有脏页记录。

注意在评测时，`include/dirtyqueue.h`、`kern/dirtyqueue.c` 都将被替换为标准版本，请不要修改这些文件的内容。

页写入异常处理

在 Lab4 课下实验中，实现了 `do_tlb_mod` 函数，其用于处理 TLB Mod 异常。在课下实现中，对于所有的 TLB Mod 异常，都统一交由 `env_user_tlb_mod_entry` 处理。但在本题目中，需要根据页表项的软件标志位来区分：应当交由 `env_user_tlb_mod_entry` 处理 CoW 逻辑，还是交由 `log_entry` 处理脏页跟踪逻辑。

修改后的 `do_tlb_mod` 的基本框架如下（`kern/tlbex.c`）：

```
void do_tlb_mod(struct Trapframe *tf) {
    Pte *pte;
    page_lookup(cur_pgdir, tf->cp0_badvaddr, &pte);
    assert((pte != NULL));

    // 保存陷阱帧，用于恢复现场。
    struct Trapframe tmp_tf = *tf;

    if ((*pte & PTE_COW) != 0) {
        // 处理 CoW 逻辑
    } else if ((*pte & PTE_LOG) != 0) {
        // 处理脏页跟踪逻辑
    } else {
        panic("Unexpected TLB Mod for va = 0x%08lx, epc = 0x%08lx, pte = 0x%08lx",
            tf->cp0_badvaddr, tf->cp0_epc, *pte);
    }
}
```

在课下实验中，触发 CoW 逻辑时，总是交由用户态处理。但在触发脏页记录时，并不总是交由用户态处理：

- （内核态）总是将触发脏页记录的页面的起始地址加入脏页队列中；
- （内核态）总是修改对应的页表项，去除 `PTE_LOG` 标志位，重新设置 `PTE_D` 标志位；
- 若**加入后**，脏页队列已满
- 若设置了用户态脏页处理逻辑
- 在用户异常栈上分配空间，分别用于存储**陷阱帧**和**所有脏页记录**，我们总是假定用户异常栈上**有足够的空间**来容纳陷阱帧和所有脏页记录；
- 将陷阱帧和所有脏页记录复制到用户异常栈上分配的空间中（使用 `dirty_remove` 可逐一获取脏页记录，该函数将同时从脏页队列中移除相应记录，故所有脏页记录获取完成后，脏页队列应当已为空）；
- 设置返回到用户态时的状态，将 `a0` 寄存器设置为陷阱帧的起始地址，`a1` 寄存器设置为脏页记录的起始地址，`cp0_epc` 寄存器设置为用户态脏页处理代码的入口地址；
- 从内核态返回。
- 若未设置用户态脏页处理逻辑，则从队列中丢弃（出队）最旧的记录，保证队列是未满足状态，直接从内核态返回即可。
- 若**加入后**，脏页队列未满足，直接从内核态返回即可。

注意上述过程实际维护了脏页队列的不变量：每次进入 `do_tlb_mod` 函数时，脏页队列都是非满的。这要求每次从 `do_tlb_mod` 函数返回时，保证脏页队列是非满的。

用户态脏页处理函数的结构如下：

```
void __attribute__((noreturn)) dirty_page_full_entry(struct Trapframe *tf, u_long
*dirty_pages) {
    ...
    // 设置陷阱帧，恢复现场
    // 该函数不应当返回
}
```

其中参数 `struct Trapframe *tf`、`u_long *dirty_pages` 由内核通过陷阱帧中的 `a0`、`a1` 寄存器传递。

- `struct Trapframe *tf` 包含了触发写入异常，进入内核态前的现场信息，用于在该处理函数结束时恢复现场；
- `u_long *dirty_pages` 是一个**一维数组**，从下标 0 开始包含了从旧到新的脏页写入记录（每个记录为**脏页的起始地址**），数组的长度为 `ENV_MAX_DIRTY_LOG_COUNT`（该宏在 `dirtyqueue.h` 中定义）

系统调用约定

你需要实现下列系统调用，它们的定义与功能如下：

1. `int sys_start_dirty_log(u_long va, u_long size)`

启动脏页追踪，将 `[va, va + size)` 范围内的所有**已映射的可写页面**加入脏页追踪，取消设置 `PTE_D` 标志，设置 `PTE_LOG` 标志，**刷新 TLB**，并返回设置了 `PTE_LOG` 标志的页面的数量。

要求 `va`、`size` 是 `PAGE_SIZE` 的整数倍，若不是，返回 `-E_INVALID`。

要求 `[va, va + size)` 范围位于合法的用户态地址范围内（`[UTEMP, UTOP)`），否则返回 `-E_INVALID`。

要求当前该用户进程并未启动脏页追踪（`log_enabled == 0`），否则返回 `-E_DIRTY_BUSY`。

若成功执行，返回设置了 `PTE_LOG` 标志的页面的数量。

2. `int sys_set_dirty_log_entry(u_long entry)`

设置用户态脏页处理函数的入口点。无论当前是否启动了脏页都可以设置。**若传入 0，表示取消设置。**

若传入的参数不是 0，则要求其必须位于合法的用户态地址范围内（`[UTEMP, UTOP)`），否则返回 `-E_INVALID`。

若成功执行，返回 0。

3. `int sys_stop_dirty_log()`

停止脏页追踪，停止追踪后，对应的追踪页应当能被正常写入，且不再产生脏页记录。需要**遍历用户进程的地址空间**（`[UTEMP, UTOP)`），对于已映射的页面，若其有 `PTE_LOG` 标志，则去除该标志，重新设置 `PTE_D` 标志，**刷新 TLB**，并返回取消设置 `PTE_LOG` 标志的页面的数量。不修改已经设置的用户态脏页处理函数的入口点。

要求当前该用户进程已经启动脏页追踪（`log_enabled == 1`），否则返回 `-E_DIRTY_OFF`。

若成功执行，返回取消设置 `PTE_LOG` 标志的页面的数量。注意设置了 `PTE_LOG` 的页面在写入一次后，将去除对应页面的 `PTE_LOG` 标志，故返回的值与调用 `sys_start_dirty_log` 时返回的值不一定相同。

4. `int sys_get_dirty_log(u_long *ptr)`

获取本进程的一项脏页记录（先进先出），将该脏页的起始地址写入到 `ptr` 指针指向的内存中。已获取的脏页记录将从脏页队列中移除。

要求 `ptr` 指针指向的内存必须位于合法的用户态地址范围内（`[UTEMP, UTOP)`），否则返回 `-E_INVAL`。

要求 `ptr` 指针指向的内存必须**已经映射且可写**，否则返回 `-E_INVAL`。

若目前本进程的脏页记录为空，则返回 `-E_DIRTY_EMPTY`，不修改 `ptr` 指向的内存。

若成功执行，将脏页的起始地址写入 `ptr` 指向的内存中，并返回 `0`。

注意事项

测试数据保证，**如果进程启动了脏页追踪，则不会进行 `fork`，也不会出现 `fork` 后的（父、子）进程启动脏页追踪的情况**。即，保证不出现一页同时有 `PTE_COW` 和 `PTE_LOG` 标记的情况。

题目要求

1. 在 `include/env.h` 的 `Env` 结构体中，添加所需的字段，注意需要导入 `dirtyqueue.h` 头文件。

```
// 新增部分开始
#include <dirtyqueue.h>
// 新增部分结束

struct Env {
    ...

    // 新增部分开始
    u_int log_enabled;
    struct DirtyPageQueue log_queue;
    u_long log_entry;
    // 新增部分结束
};
```

2. 在初始化 `Env` 结构体时，初始化本题目所需的结构体字段，注意需要导入 `dirtyqueue.h` 头文件。（`kern/env.c`，`env_alloc` 函数）

```
// 新增部分开始
#include <dirtyqueue.h>
// 新增部分结束

int env_alloc(struct Env **new, u_int parent_id) {

    ...
    // Lab4-Extra: Your code here. (1 / 19)
    // 1. 初始化 log_enabled 字段
    // 2. 初始化 log_queue 字段
    // HINT: 你可以使用 `dirty_init` 函数
    // 3. 初始化 log_entry 字段
    ...
}
```

3. 在 `include/mmu.h` 中添加 `PTE_LOG` 定义。

```
// Dirty page log. Reserved for software.
#define PTE_LOG 0x0004
```

4. 在 `include/syscall.h` 中, 添加相关系统调用的枚举, 请添加到 `MAX_SYSNO` 之前。

```
enum {
    ...
    SYS_read_dev,

    // 新增部分开始
    SYS_start_dirty_log,
    SYS_stop_dirty_log,
    SYS_set_dirty_log_entry,
    SYS_get_dirty_log,
    // 新增部分结束

    MAX_SYSNO,
};
```

5. 在 `include/error.h` 中, 添加所需的错误码。

```
// 尝试在目标进程已启用脏页追踪的情况下, 再次启动脏页追踪
#define E_DIRTY_BUSY 100
// 尝试在目标进程未启用脏页追踪的情况下, 停止脏页追踪
#define E_DIRTY_OFF 101
// 尝试在脏页队列为空的情况下, 获取脏页记录
#define E_DIRTY_EMPTY 102
```

6. 在 `kern/syscall_all.c` 中, 在 `syscall_table` 中注册相关系统调用, 下文将详细阐述系统调用的实现方式并提供部分代码, 你不需要完全从头开始实现这些系统调用, 注意需要导入 `dirtyqueue.h` 头文件。

```
// 新增部分开始
#include <dirtyqueue.h>
// 新增部分结束

// 新增部分开始
int sys_start_dirty_log(u_long va, u_long size) {}
int sys_set_dirty_log_entry(u_long entry) {}
int sys_stop_dirty_log() {}
int sys_get_dirty_log(u_long *ptr) {}
// 新增部分结束

void *syscall_table[MAX_SYSNO] = {
    ...
    [SYS_read_dev] = sys_read_dev,

    // 新增部分开始
    [SYS_start_dirty_log] = sys_start_dirty_log,
    [SYS_stop_dirty_log] = sys_stop_dirty_log,
    [SYS_set_dirty_log_entry] = sys_set_dirty_log_entry,
    [SYS_get_dirty_log] = sys_get_dirty_log,
    // 新增部分结束
```

```
};
```

7. 在 `user/include/lib.h` 中添加如下定义:

```
void __attribute__((noreturn)) dirty_page_full_entry(struct Trapframe *tf, u_long
*dirty_pages);
int syscall_start_dirty_log(u_long va, u_long size);
int syscall_set_dirty_log_entry(void (*func_ptr)(struct Trapframe *tf, u_long
*dirty_pages));
int syscall_stop_dirty_log();
int syscall_get_dirty_log(u_long *ptr);
```

8. 在 `user/lib/syscall_lib.c` 中添加实现, 下面的代码中**不含**你需要实现的内容:

```
int syscall_start_dirty_log(u_long va, u_long size) {
    return msyscall(SYS_start_dirty_log, va, size);
}

int syscall_set_dirty_log_entry(void (*func_ptr)(struct Trapframe *tf, u_long
*dirty_pages)) {
    return msyscall(SYS_set_dirty_log_entry, func_ptr);
}

int syscall_stop_dirty_log() {
    return msyscall(SYS_stop_dirty_log);
}

int syscall_get_dirty_log(u_long *ptr) {
    return msyscall(SYS_get_dirty_log, ptr);
}

void __attribute__((noreturn)) dirty_page_full_entry(struct Trapframe *tf,
u_long *dirty_pages) {
    debugf("[dirty_page_full_entry] tf at 0x%08lx dirty_pages at 0x%08lx\n", tf,
dirty_pages);
    debugf("[dirty_page_full_entry] Dumping dirty pages...\n");

    for (int i = 0; i < ENV_MAX_DIRTY_LOG_COUNT; i += 1) {
        debugf("[dirty_page_full_entry] [%02d] = [%08lx]\n", i, dirty_pages[i]);
    }

    debugf("[dirty_page_full_entry] Dumping complete.\n");

    int r = syscall_set_trapframe(0, tf);
    user_panic("syscall_set_trapframe returned %d", r);
}
```

9. 在 `kern/syscall_all.c` 中, 实现 `sys_start_dirty_log` 系统调用:

```
int sys_start_dirty_log(u_long va, u_long size) {
    // 1. 检查 `va`、`size` 是否是 `PAGE_SIZE` 的整数倍
    // Lab4-Extra: Your code here. (2 / 19)

    // 2. 检查虚拟地址范围的合法性
```

```
// HINT: 你可以使用 `is_illegal_va_range` 函数
// Lab4-Extra: Your code here. (3 / 19)

// 3. 检查是否已经启动脏页记录
// HINT: 可以使用 `curenv` 访问当前进程的进程控制块
// Lab4-Extra: Your code here. (4 / 19)

// 4. 标记启动脏页记录
// Lab4-Extra: Your code here. (5 / 19)

// 记录设置了 `PTE_LOG` 标志的页面的数量
int log_page_count = 0;

// 5. 遍历 [va, va + size) 范围内的页表项, 若映射有效且可写, 则去除可写标记, 加上追踪标记
// 不要忘记更新 `log_page_count`!
// 不要忘记维护 TLB: 在更新页表项后, 需要使用 `tlb_invalidate` 函数将 TLB 中的旧映射无效化
// Lab4-Extra: Your code here. (6 / 19)

return log_page_count;
}
```

10. 在 `kern/syscall_all.c` 中, 实现 `sys_set_dirty_log_entry` 系统调用:

```
int sys_set_dirty_log_entry(u_long entry) {
// 1. 判断 `entry` 对应的虚拟地址是否合法, 判断 `entry` 是否为 `0` (表示取消设置)
// HINT: 你可以使用 `is_illegal_va` 函数
// Lab4-Extra: Your code here. (7 / 19)

// 2. 在进程控制块中设置用户态脏页处理函数的入口点
// HINT: 可以使用 `curenv` 访问当前进程的进程控制块
// Lab4-Extra: Your code here. (8 / 19)

return 0;
}
```

11. 在 `kern/syscall_all.c` 中, 实现 `sys_stop_dirty_log` 系统调用:

```
int sys_stop_dirty_log() {
// 1. 检查是否已经启动脏页记录
// HINT: 可以使用 `curenv` 访问当前进程的进程控制块
// Lab4-Extra: Your code here. (9 / 19)

// 2. 标记停止脏页记录
// Lab4-Extra: Your code here. (10 / 19)

// 记录取消设置 `PTE_LOG` 标志的页面的数量
int log_page_count = 0;

// 3. 遍历进程的页表项 (`[UTEMP, UTOP)`), 若映射有效且已标记为追踪页, 则去除追踪标记, 加上可写标记
// 不要忘记更新 `log_page_count`!
// 不要忘记维护 TLB: 在更新页表项后, 需要使用 `tlb_invalidate` 函数将 TLB 中的旧映射无效化
// Lab4-Extra: Your code here. (11 / 19)
```



```
return log_page_count;
}
```

12. 在 `kern/syscall_all.c` 中, 实现 `sys_get_dirty_log` 系统调用:

```
int sys_get_dirty_log(u_long *ptr) {
// 1. 检查 `ptr` 指向的虚拟地址的合法性
// HINT: 你可以使用 `is_illegal_va` 函数
// Lab4-Extra: Your code here. (12 / 19)

// 2. 检查 `ptr` 指向的虚拟地址是否已有映射, 且可写
Pte *pte = NULL;
if (page_lookup(cur_pgdir, (u_long)ptr, &pte) == NULL) {
return -E_INVALID;
}

if (!((*pte & PTE_V) && (*pte & PTE_D))) {
return -E_INVALID;
}

// 3. 检查脏页队列是否为空
// Lab4-Extra: Your code here. (13 / 19)

// 4. 从队列中出队一个脏页记录
// Lab4-Extra: Your code here. (14 / 19)
u_long dirty_page_addr = ??;

// 5. 向 `ptr` 指向的位置写入取出的脏页记录
*ptr = dirty_page_addr;

return 0;
}
```

13. 在 `kern/tlbex.c` 中, 修改 `do_tlb_mod`, 实现脏页记录逻辑, 请将课下实现的 `do_tlb_mod` 函数**完全替换**为下面给出的版本。注意需要导入 `dirtyqueue.h` 头文件:

```
// 新增部分开始
#include <dirtyqueue.h>
// 新增部分结束

void do_tlb_mod(struct Trapframe *tf) {
// 1. 获取发生页写入异常的页面
Pte *pte;
page_lookup(cur_pgdir, tf->cp0_badvaddr, &pte);
assert((pte != NULL));

// 2. 保存异常现场 (陷阱帧)
struct Trapframe tmp_tf = *tf;

if ((*pte & PTE_COW) != 0) {
// 3. 处理 Cow 页面
if (tf->regs[29] < USTACKTOP || tf->regs[29] >= UXSTACKTOP) {
tf->regs[29] = UXSTACKTOP;
}
```

```

}

tf->regs[29] -= sizeof(struct Trapframe);
*(struct Trapframe *)tf->regs[29] = tmp_tf;

if (curenv->env_user_tlb_mod_entry) {
tf->regs[4] = tf->regs[29];
tf->regs[29] -= sizeof(tf->regs[4]);
// Hint: Set 'cp0_epc' in the context 'tf' to 'curenv->env_user_tlb_mod_entry'.
tf->cp0_epc = curenv->env_user_tlb_mod_entry;
} else {
panic("TLB Mod but no user handler registered");
}
} else if ((*pte & PTE_LOG) != 0) {
// 4. 处理脏页追踪页面
// Precondition: `log_queue` 非满
assert(!dirty_is_full(&curenv->log_queue));

// 5. 去除触发页面的脏页追踪标志, 重新添加可写标志
// Lab4-Extra: Your code here. (15 / 19)

// 6. 刷新 TLB: 在更新页表项后, 需要使用 `tlb_invalidate` 函数将 TLB 中的旧映射无效化
// Lab4-Extra: Your code here. (16 / 19)

// 7. 将脏页的起始地址添加到脏页队列中
// HINT: 使用 `ROUNDDOWN(a, n)` 宏, 可以将整数 a 向下对齐到整数 n, 该宏返回对齐后的值
// Lab4-Extra: Your code here. (17 / 19)

// 8. 若队列已满
if (dirty_is_full(&curenv->log_queue)) {
// 是否设置了用户态脏页处理程序?
if (curenv->log_entry != 0) {
// 若队列已满, 且已经设置用户态脏页处理程序

// 9. 设置异常栈指针
if (tf->regs[29] < USTACKTOP || tf->regs[29] >= UXSTACKTOP) {
tf->regs[29] = UXSTACKTOP;
}

// 10. 在异常栈上分配保存陷阱帧的空间, 并写入陷阱帧
tf->regs[29] -= sizeof(struct Trapframe);
*(struct Trapframe *)tf->regs[29] = tmp_tf;

// 11. 记录保存的陷阱帧的起始地址
u_long user_tf_addr = tf->regs[29];

// 12. 在异常栈上分配保存脏页记录的空间
// HINT: 我们总是假定用户异常栈上有足够的空间来容纳陷阱帧和所有脏页记录
// HINT: 你可以参考“10. 在异常栈上分配保存陷阱帧的空间”的方式进行分配, 注意分配的空间应当能保
存下所有脏页
// HINT: 脏页队列的容量为 ENV_MAX_DIRTY_LOG_COUNT (dirtyqueue.h)
// HINT: 将脏页记录按照一维数组的形式复制到用户异常栈, 每个元素的大小为 sizeof(u_long)
// Lab4-Extra: Your code here. (18 / 19)

// 13. 记录脏页记录的起始地址
u_long user_log_addr = tf->regs[29];

```

```

// 14. 逐一将脏页记录出队，并写入异常栈上分配的空间中
// HINT: 代码执行到此处，前提条件是脏页队列已满
// HINT: 脏页队列的容量为 ENV_MAX_DIRTY_LOG_COUNT (dirtyqueue.h)
// Lab4-Extra: Your code here. (19 / 19)

// 15. 设置陷阱帧的 a0 寄存器为保存的陷阱帧的起始地址
tf->regs[4] = user_tf_addr;
// 16. 设置陷阱帧的 a1 寄存器为脏页记录的起始地址
tf->regs[5] = user_log_addr;

tf->regs[29] -= 2 * sizeof(tf->regs[4]);
// 17. 设置返回地址
tf->cp0_epc = curenv->log_entry;
} else {
// 若队列已满，且未经设置用户态脏页处理程序
// 18. 从脏页队列中出队一个脏页记录（并丢弃），保证返回时脏页队列非满
dirty_remove(&curenv->log_queue);
}
}
// Postcondition: `log_queue` 非满

} else {
panic("Unexpected TLB Mod for va = 0x%08lx, epc = 0x%08lx, pte = 0x%08lx", tf-
>cp0_badvaddr, tf->cp0_epc, *pte);
}
}
}

```

样例输出 & 本地测试

对于如下用户程序样例：

```

#include <error.h>
#include <lib.h>
#include <limits.h>

#define RW_PAGES_SIZE 64
#define RO_PAGES_SIZE 32

#define WRITE_PAGES_COUNT 43

#define MAX_COLLECT_PAGES 64

#define BEGIN_ADDR 0x50000000

int main() {
    debugf("BEGIN OF TEST\n");
    // 准备用于测试的可写页与只读页
    u_long rw_pages[RW_PAGES_SIZE] = {0};
    u_long ro_pages[RO_PAGES_SIZE] = {0};

    for (int i = 0; i < RW_PAGES_SIZE; i += 1) {
        u_long current_addr = BEGIN_ADDR + i * 0x2000;

        rw_pages[i] = current_addr;
    }
}

```

```

}

// 测试对非页面边界地址的写入，检查记录脏页时是否向下对齐到页面边界
rw_pages[1] += 0x0004;
rw_pages[2] += 0x0008;

for (int i = 0; i < RO_PAGES_SIZE; i += 1) {
    u_long current_addr = BEGIN_ADDR + i * 0x2000 + 0x1000;

    ro_pages[i] = current_addr;
}

u_long min_addr = ULONG_MAX;
u_long max_addr = 0;

for (int i = 0; i < RW_PAGES_SIZE; i += 1) {
    u_long current_page = rw_pages[i];

    if (current_page < min_addr) {
        min_addr = current_page;
    }

    if (current_page > max_addr) {
        max_addr = current_page;
    }
}

for (int i = 0; i < RO_PAGES_SIZE; i += 1) {
    u_long current_page = ro_pages[i];

    if (current_page < min_addr) {
        min_addr = current_page;
    }

    if (current_page > max_addr) {
        max_addr = current_page;
    }
}

debugf("min addr = 0x%08lx, max addr = 0x%08lx\n", min_addr, max_addr);

// 取消地址范围内的映射
for (u_long current_va = min_addr; current_va <= (max_addr); current_va +=
PAGE_SIZE) {
    panic_on(syscall_mem_unmap(0, (void *)current_va));
}

// 分配可写页
for (int i = 0; i < RW_PAGES_SIZE; i += 1) {
    panic_on(syscall_mem_alloc(0, (void *)rw_pages[i], PTE_V | PTE_D));
}

// 分配只读页
for (int i = 0; i < RO_PAGES_SIZE; i += 1) {
    panic_on(syscall_mem_alloc(0, (void *)ro_pages[i], PTE_V));
}

```

```

u_long size = max_addr - min_addr + PAGE_SIZE;

debugf("[1 / 4] OK: Preparation\n");

// 开启脏页记录
int ret = syscall_start_dirty_log(min_addr, size);

if (ret != RW_PAGES_SIZE) {
    user_panic("ERROR: unexpected ret value from syscall_start_dirty_log =
%d\n", ret);
} else {
    debugf("[2 / 4] OK: syscall_start_dirty_log\n");
}

// 设置用户脏页处理函数
ret = syscall_set_dirty_log_entry(dirty_page_full_entry);

if (ret != 0) {
    user_panic("ERROR: unexpected ret value from syscall_set_dirty_log_entry
= %d", ret);
} else {
    debugf("[3 / 4] OK: syscall_set_dirty_log_entry\n");
}

// 写入被追踪的页面
for (int i = 0; i < WRITE_PAGES_COUNT; i += 1) {
    *(int *)rw_pages[i] = i;
}

debugf("[4 / 4] OK: write logged page\n");

int actual_log_entry_count = 0;

// 读取未被用户态处理的剩余页面
debugf("Remaining entries: \n");
while (1) {
    u_long current_entry = 0;
    int ret = syscall_get_dirty_log(&current_entry);

    if (ret == -E_DIRTY_EMPTY) {
        break;
    }

    if (ret < 0) {
        user_panic("ERROR: unexpected return value from syscall_get_dirty_log
= %d", ret);
        break;
    }

    debugf("[%02d] = [0x%08lx]\n", actual_log_entry_count, current_entry);

    actual_log_entry_count += 1;

    if (actual_log_entry_count >= MAX_COLLECT_PAGES) {
        break;
    }
}

```

```

    }
}

debugf("END OF TEST\n");

return 0;
}

```

应当输出:

```

BEGIN OF TEST
min addr = 0x50000000, max addr = 0x5007e000
[1 / 4] OK: Preparation
[2 / 4] OK: syscall_start_dirty_log
[3 / 4] OK: syscall_set_dirty_log_entry
[dirty_page_full_entry] tf at 0x7f3fff68 dirty_pages at 0x7f3ffee8
[dirty_page_full_entry] Dumping dirty pages...
[dirty_page_full_entry] [00] = [50000000]
[dirty_page_full_entry] [01] = [50002000]
[dirty_page_full_entry] [02] = [50004000]
[dirty_page_full_entry] [03] = [50006000]
[dirty_page_full_entry] [04] = [50008000]
[dirty_page_full_entry] [05] = [5000a000]
[dirty_page_full_entry] [06] = [5000c000]
[dirty_page_full_entry] [07] = [5000e000]
[dirty_page_full_entry] [08] = [50010000]
[dirty_page_full_entry] [09] = [50012000]
[dirty_page_full_entry] [10] = [50014000]
[dirty_page_full_entry] [11] = [50016000]
[dirty_page_full_entry] [12] = [50018000]
[dirty_page_full_entry] [13] = [5001a000]
[dirty_page_full_entry] [14] = [5001c000]
[dirty_page_full_entry] [15] = [5001e000]
[dirty_page_full_entry] [16] = [50020000]
[dirty_page_full_entry] [17] = [50022000]
[dirty_page_full_entry] [18] = [50024000]
[dirty_page_full_entry] [19] = [50026000]
[dirty_page_full_entry] [20] = [50028000]
[dirty_page_full_entry] [21] = [5002a000]
[dirty_page_full_entry] [22] = [5002c000]
[dirty_page_full_entry] [23] = [5002e000]
[dirty_page_full_entry] [24] = [50030000]
[dirty_page_full_entry] [25] = [50032000]
[dirty_page_full_entry] [26] = [50034000]
[dirty_page_full_entry] [27] = [50036000]
[dirty_page_full_entry] [28] = [50038000]
[dirty_page_full_entry] [29] = [5003a000]
[dirty_page_full_entry] [30] = [5003c000]
[dirty_page_full_entry] [31] = [5003e000]
[dirty_page_full_entry] Dumping complete.
[4 / 4] OK: write logged page
Remaining entries:
[00] = [0x50040000]
[01] = [0x50042000]
[02] = [0x50044000]

```

```
[03] = [0x50046000]
[04] = [0x50048000]
[05] = [0x5004a000]
[06] = [0x5004c000]
[07] = [0x5004e000]
[08] = [0x50050000]
[09] = [0x50052000]
[10] = [0x50054000]
END OF TEST
[00000800] destroying 00000800
[00000800] free env 00000800
i am killed ...
panic at sched.c:46 (schedule): schedule: no runnable envs
```

样例说明

1. 在用户进程 `[0x50000000, 0x5007e000]` 范围内映射 64 个可写页面，32 个只读页面，在该范围内启用脏页记录，记录对于可写页面的写入操作；
2. 测试程序对从 `0x50000000` 开始的可写页面依次写入，内核逐个记录对脏页的写入，当脏页记录数量达到脏页队列容量 32 时，调用用户态脏页处理函数。脏页处理函数依次打印 32 个脏页记录（从 `0x50000000` 到 `0x5003e000`，共 32 个记录，输出中以 `[dirty_page_full_entry]` 开头）；
3. 共对 43 个可写页面进行写入，当写入 32 个可写页面时，触发用户态脏页处理，剩余的 11 次写入继续记录在脏页队列中，通过 `syscall_get_dirty_log` 依次获取，打印出从 `0x50040000` 到 `0x50054000` 的 11 个记录（从输出中 `Remaining entries:` 开始的 11 行）；
4. 注意程序中实际写入了地址 `0x50002004`、`0x50004008`，但脏页记录应当对齐页面边界，记录为 `0x50002000`、`0x50004000`（输出中 `[dirty_page_full_entry] [01]`、`[dirty_page_full_entry] [02]` 条目）。

你可以使用

- `make test lab=4_dirty_page_sample && make run` 在本地测试上述样例（调试模式）
- `MOS_PROFILE=release make test lab=4_dirty_page_sample && make run` 在本地测试上述样例（开启优化）

提交评测 & 评测标准

请在开发机中执行下列命令后，**在课程网站上提交评测**。

```
$ cd ~/学号
$ git add -A
$ git commit -m "message" # 请将 message 改为有意义的信息
$ git push
```

在线评测时，所有的 `.mk` 文件、所有的 `Makefile` 文件、`init/init.c`、`include/dirtyqueue.h`、`kern/dirtyqueue.c` 以及 `tests/` 和 `tools/` 目录下的所有文件都可能被替换为标准版本，因此请同学们在本地开发时，**不要**在这些文件中编写实际功能所依赖的代码。

测试点和分数说明如下：

测试点序号	评测内容	分数
1	样例	3
2	<code>sys_start_dirty_log</code> 的前置条件（地址合法性、是否启动脏页记录）	5
3	<code>sys_start_dirty_log</code> 对页表的修改、返回值	5
4	<code>sys_start_dirty_log</code> 维护 TLB	5
5	<code>sys_stop_dirty_log</code> 的前置条件（是否启动脏页记录）	5
6	<code>sys_stop_dirty_log</code> 对页表的修改、返回值	5
7	<code>sys_stop_dirty_log</code> 维护 TLB	5
8	<code>sys_set_dirty_log_entry</code> 设置用户脏页处理函数	10
9	<code>sys_get_dirty_log</code> 的前置条件（地址合法性、队列是否为空）	5
10	<code>sys_get_dirty_log</code> 应以先进先出的方式返回脏页记录	5
11	<code>do_tlb_mod</code> 处理页写入异常（脏页队列未滿）	5
12	<code>do_tlb_mod</code> 处理页写入异常（脏页队列已滿）	5
13	用户态调用有关系统调用（弱测）	10
14	用户态测试脏页记录（ <code>syscall_get_dirty_log</code> ）	10
15	综合测试	17

提示：本题测试点独立性相对较高，且不依赖样例。若未完全实现，也可以提交以获得部分分数。

测试点依赖关系如下（箭头方向表示依赖方向）：

